my-agents-cli-mod

`agents-cli create my-agents-cli-mod` (agents-cli **1.13.0**, 템플릿 `adk_python`)로 생성한 에이전트 프로젝트에서 `app/agent.py`를 수정해 **전세계 도시의 실시간 현지 시각·날씨를 알려주고 일반 질문에도 답하는 범용 AI 어시스턴트**로 만든 실습 프로젝트입니다.

> 영어판: README-EN.md · 템플릿 기본 안내(영문 요약): README.md

---

1. 원본 템플릿 대비 변경 사항 (`app/agent.py`)

항목	내용
에이전트 이름	`root_agent.name = "my_agents_cli_mod"` (`agents-cli-manifest.yaml`의 `root_agent_name`과 동일하게 유지), `App(name="app")`
모델	`ADK_MODEL` 환경 변수 (기본값 `gemini-3.8-flash`), `HttpRetryOptions(attempts=3)` 재시도
도구	`geocode` · `get_current_time` · `get_weather` (3개)
외부 API	Open-Meteo Geocoding / Forecast API — **API 키 불필요**
한국어 지원	한글 도시명 → 영문 변환 사전(`KOREAN_CITY_MAP`, 서울·부산·도쿄·뉴욕 등 28개), WMO 기상 코드 한국어 매핑(`WMO_WEATHER_CODES`)
오프라인 대비	네트워크 실패 또는 `AGENT_OFFLINE=1` 일 때 주요 도시 좌표·시간대 백업(`_OFFLINE`)과 기본 날씨 응답(`source: offline_fallback`) 사용
지시문	시간·날씨 질문은 도구를 사용해 한국어로 답변, 그 외 일반 질문(상식·프로그래밍·번역 등)은 도구 없이 직접 답변

도구 요약

도구	입력	반환
`geocode(location)`	도시/지역 이름 (한글·영문)	`display_name`, `latitude`, `longitude`, `timezone`
`get_current_time(location, query, latitude, longitude)`	도시 이름 또는 좌표	`%Y-%m-%d %H:%M:%S %Z (UTC%z)`
`get_weather(location, query, latitude, longitude)`	도시 이름 또는 좌표	날씨 상태, 기온, 체감 온도, 습도, 풍속

---

2. 환경 변수 (`.env`)

`app/agent.py` 는 `.env` 를 두 곳에서 읽습니다:

1. `my-agents-cli-mod/.env` (있으면 우선 적용, `override=True`)
2. 실습 루트 `ADK_Labs/.env` (`_ROOT.parent.parent / ".env"`)

따라서 **루트 `ADK_Labs/.env` 하나만 설정해도** 이 프로젝트가 동작합니다. 이 폴더에는 별도 `.env` 가 없습니다.

```
# ① Google AI Studio
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=<your-gemini-api-key>

# ② Vertex AI (gcloud auth application-default login 필요)
# GOOGLE_GENAI_USE_VERTEXAI=TRUE
# GOOGLE_CLOUD_PROJECT=<your-gcp-project-id>
# GOOGLE_CLOUD_LOCATION=global

# (선택) 모델 변경 / 오프라인 모드
# ADK_MODEL=gemini-3.5-flash-lite
# AGENT_OFFLINE=1
```

- `GOOGLE_GENAI_USE_VERTEXAI=TRUE` 이면 `GOOGLE_API_KEY` · `GEMINI_API_KEY` 를 제거하고 `GOOGLE_CLOUD_LOCATION` 기본값을 `global` 로 설정합니다.
- 템플릿이 만든 `.env.example` 은 `GOOGLE_GENAI_USE_ENTERPRISE` / `GEMINI_API_KEY` 형식을 사용합니다. 이 형식을 쓰려면 이 폴더에 `.env` 로 복사해 사용합니다.

3. 요구 사항

- **uv**: Python 패키지 관리자 — 설치 안내
- **agents-cli**: `uv tool install google-agents-cli` (또는 루트 `requirements.txt` 설치 시 함께 설치됨)
- **Google Cloud SDK** (배포 시): 설치 안내
- `a2a-sdk[http-server]>=1.0.0,<2` 는 `pyproject.toml` 에 이미 포함되어 있습니다.

4. 실행 방법

`part03_cli/my-agents-cli-mod` 디렉터리에서 실행합니다:

```
cd part03_cli/my-agents-cli-mod

# 1. 의존성 설치 (.venv / uv.lock 생성)
agents-cli install

# 2. 로컬 플레이그라운드 웹 UI 실행
agents-cli playground

# 3. FastAPI 서버 직접 실행 (웹 UI + REST/SSE + A2A)
uv run uvicorn app.fast_api_app:app --host 0.0.0.0 --port 8000 --reload

# 4. ADK CLI로 실행 (에이전트 폴더 = app)
uv run adk web --port 8008
uv run adk run app

# 5. 오프라인 모드(외부 날씨 API 호출 없이 백업 데이터 사용) - PowerShell
$env:AGENT_OFFLINE="1"; agents-cli playground
```

질문 예시:

- `지금 서울 몇 시야?` · `도쿄랑 뉴욕 날씨 비교해줘` · `파리의 체감 온도는?`
- `파이썬 리스트 컴프리헨션 설명해줘` (도구 호출 없이 답변)

5. 테스트 · 평가 · 배포

명령어	설명
`agents-cli lint`	코드 품질 검사 (ruff · ty · codespell)
`uv run pytest tests/unit`	단위 테스트 (`test_dummy.py` , LLM 불필요)
`uv run pytest tests/integration`	통합 테스트 (`test_agent.py` : Runner 스트리밍, `test_server_e2e.py` : FastAPI·A2A E2E) — Gemini 키 필요
`agents-cli eval run`	`tests/eval/datasets/basic-dataset.json` (인사·샌프란시스코 날씨·프랑스 수도 3건)으로 실행 후 `custom_response_quality` (LLM-as-judge, 1~5점) 채점
`agents-cli deploy`	Google Cloud **Agent Runtime**에 배포 (`deployment_target: agent_runtime` , 리전 `us-east1`) → `deployment_metadata.json` 기록
`agents-cli publish gemini`	배포된 에이전트를 Gemini에 등록
`agents-cli scaffold enhance`	CI/CD 및 Terraform 코드 추가 (본 프로젝트의 `deployment/terraform/` 이 이 명령으로 생성됨)
`agents-cli infra cicd`	CI/CD 파이프라인 및 인프라 원클릭 설정
`agents-cli scaffold upgrade`	사용자 수정 사항을 보존하면서 템플릿 업그레이드

```
gcloud config set project <your-project-id>
agents-cli deploy --no-confirm-project
```

> COPY ./pyproject.toml ./README.md ./uv.lock*
> ./ , README.md

6. 서버 구조 (`app/fast_api_app.py`)

- `get_fast_api_app(agents_dir=..., web=True, ...)` 로 ADK 웹 UI와 `/run` , `/run_sse` 등 기본 API 라우트를 생성합니다.
- Lifespan에서 `Runner(app=adk_app, auto_create_session=True)` 를 만들고 `attach_a2a_routes()` 로 **A2A JSON-RPC 엔드포인트 `/a2a/app` **을 연결합니다 (`is_a2a: true`).
- `attach_reasoning_engine_routes(app)` 로 Cloud 콘솔 플레이그라운드용 `reasoning_engine` 프록시 라우트를 추가합니다.
- 세션은 `in_memory` (manifest `session_type`), 텔레메트리는 `GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY=true` 일 때 Cloud Trace 등으로 내보냅니다.
- 직접 실행 시 포트 `8000` , 컨테이너(Dockerfile)에서는 포트 `8080` 을 사용합니다.

A2A 상호 운용성 테스트에는 A2A Inspector를 사용합니다.

---

7. 부가 자료: ADK Web 아키텍처 슬라이드

파일	설명
`generate_adk_web_ppt.py`	`python-pptx` 로 ADK Web 아키텍처 설명 슬라이드(8장)를 생성하는 스크립트
`ADK_Web_Architecture_Guide.pptx`	위 스크립트의 출력물 — 전체 아키텍처, 웹 UI 패키징·FastAPI 마운트, 서버 기동 라이프사이클, `/run_sse` 스트리밍, 개발자 도구, `adk web` vs `adk api_server` 비교, 실행 옵션 요약

```
uv pip install python-pptx
uv run python generate_adk_web_ppt.py                  # 기본 출력: ADK_Web_Architecture_Guide.pptx
uv run python generate_adk_web_ppt.py my_slides.pptx   # 출력 파일명 지정
```

---

8. 📂 상세 폴더 구조 및 생성 시점

```
my-agents-cli-mod/
├── 📁 app/                                  # 🟢 핵심 에이전트 애플리케이션 (agent_directory)
│   ├── 📄 __init__.py                       # 🟢 Python 패키지 선언
│   ├── 📄 agent.py                          # 🖍️ 실시간 시각·날씨 도구 + 범용 어시스턴트 (수정됨)
│   ├── 📄 fast_api_app.py                   # 🟢 웹 UI + REST/SSE + A2A FastAPI 서버
│   ├── 📁 app_utils/                        # 🟢 보조 유틸리티
│   │   ├── 📄 a2a.py                        # 🟢 A2A 라우트 연결 및 Agent Card 처리
│   │   ├── 📄 reasoning_engine_adapter.py   # 🟢 Cloud 콘솔 플레이그라운드용 reasoning_engine 프록시
│   │   └── 📄 services.py                   # 🟢 세션·아티팩트 서비스 팩토리
│   ├── 📁 .adk/                             # ⚡ [런타임] 로컬 실행 시 생성
│   │   └── 📄 session.db                    # ⚡ 세션·대화 상태 SQLite DB
│   └── 📁 __pycache__/                      # ⚡ [컴파일] 바이트코드 캐시
│
├── 📁 deployment/terraform/                 # ⚡ [scaffold enhance] GCP 인프라 Terraform 코드
│   ├── 📁 shared/                           # ⚡ completions.sql, genai_logs_schema.json,
dummy_source.b64
│   └── 📁 single-project/                   # ⚡ apis · iam · service · storage · telemetry .tf,
vars/env.tfvars
│
├── 📁 tests/                                # 🟢 테스트 및 평가
│   ├── 📁 unit/test_dummy.py                # 🟢 기본 단위 테스트
│   ├── 📁 integration/
│   │   ├── 📄 test_agent.py                 # 🟢 Runner SSE 스트리밍 통합 테스트
│   │   └── 📄 test_server_e2e.py            # 🟢 FastAPI·A2A 엔드포인트 E2E 테스트
│   └── 📁 eval/
│       ├── 📄 eval_config.yaml              # 🟢 평가 지표 설정 (custom_response_quality,
agent_turn_count)
│       ├── 📄 response_quality.py           # 🟢 LLM-as-judge 응답 품질 채점기
│       └── 📁 datasets/
│           ├── 📄 basic-dataset.json        # 🟢 평가 질의 3건
│           └── 📄 README.md                 # 🟢 평가 데이터셋 사용법
│
├── 📁 .venv/                                # ⚡ [install] agents-cli install / uv sync 가상환경
├── 📄 uv.lock                               # ⚡ [install] 패키지 버전 잠금 파일
├── 📄 deployment_metadata.json              # ⚡ [deploy] 배포된 Agent Runtime(reasoningEngines) ID·시
각 기록
│
├── 📄 agents-cli-manifest.yaml              # 🟢 agents-cli 메타데이터 (템플릿·에이전트 이름·리전·배포 대
상)
├── 📄 pyproject.toml                        # 🟢 의존성 및 ruff/ty/pytest 설정
├── 📄 Dockerfile                            # 🟢 컨테이너 빌드 명세 (python:3.12-slim, 포트 8080)
├── 📄 .env.example                          # 🟢 환경 변수 템플릿
├── 📄 .gitignore / .gcloudignore            # 🟢 Git / GCP 업로드 제외 규칙
├── 📄 GEMINI.md                             # 🟢 코딩 에이전트(Antigravity/Gemini)용 개발 가이드
├── 📄 README.md                             # 🖍️ 프로젝트 요약 (영문, Dockerfile 빌드에 필요)
│
├── 📄 generate_adk_web_ppt.py               # 🖍️ [사용자 추가] ADK Web 아키텍처 슬라이드 생성 스크립트
├── 📄 ADK_Web_Architecture_Guide.pptx       # 🖍️ [사용자 추가] 위 스크립트로 생성한 슬라이드
├── 📄 README-KR.md / README-KR.pdf          # 🖍️ [사용자 추가] 한글 상세 가이드 (본 파일)
└── 📄 README-EN.md / README-EN.pdf          # 🖍️ [사용자 추가] 영문 상세 가이드
```

아이콘	분류	설명	대표 파일
🟢	**초기 스캐폴딩**	`agents-cli create` 시 템플릿에서 자동 생성	`app/fast_api_app.py`, `tests/`, `Dockerfile`, `agents-cli-manifest.yaml`
⚡	**동적 생성**	`scaffold enhance deploy` 실행 · `pytest` 시 사후 생성	`.venv/`, `uv.lock`, `deployment/`, `deployment_metadata.json`, `app/.adk/session.db`
✏️	**사용자 수정/추가**	템플릿을 바탕으로 직접 편집하거나 추가한 파일	`app/agent.py`, `README*.md`, `generate_adk_web_ppt.py`, `ADK_Web_Architecture_Guide.pptx`